Business

Date – 16 March 2026

Name	Nivedita Tapan Biswas
Roll No	150096724019
Cohort	MZ
Case study 101	Smart Factory Automation system at AutoSmart Manufacturing Pvt.Ltd

Background:

AutoSmart Manufacturing Pvt. Ltd. is a Pune-based company producing automotive components for major automobile manufacturers. Currently, factory operations such as machine monitoring, production tracking, and quality inspection are mostly manual or semi-automated.

This results in machine downtime, human errors, inconsistent quality, and production delays.

The company spends around ₹6 Crores annually on production operations, maintenance, and quality control, but still faces inefficiencies that reduce overall productivity.

To improve performance, the company plans to implement a Smart Factory Automation system using IoT sensors, AI-based monitoring tools, and automated control systems.

These technologies will monitor machine health in real time, detect faults early, automate quality inspection, and optimize production schedules.

The company plans to invest ₹12 Crores in automation equipment, IoT integration, AI software, cloud infrastructure, and employee training.

The estimated annual operational and maintenance cost of the automation system is ₹3 Crores.

The company expects automation to improve production efficiency by 30% and reduce operational and maintenance losses by ₹4 Crores annually. Improved product quality and timely delivery can strengthen relationships with automotive clients and enhance Customer Lifetime Value (CLV).

However, the success of the project depends on system reliability, smooth integration with existing processes, and effective workforce training.

Students must analyze both financial feasibility and technical viability to evaluate whether Implementing the smart factory automation system is profitable and sustainable.

Questions for Students :

Business and Financial Analysis:

1. Calculate the total annual cost savings if automation reduces operational losses by ₹4 Crores per year.

2. Calculate the Net Benefit using: Net Benefit = Annual

Cost Savings – Annual Operational Cost – Initial Investment.

Technical and Data Analysis:

3. What production and machine variables (such as machine uptime, defect rate, production output, and downtime duration) should be collected to evaluate automation effectiveness? Explain their importance.

4. How can synthetic production and machine performance data be generated using Python?

CRM and Business Impact:

5. How do improved product quality, reduced downtime, and higher production efficiency affect client satisfaction, retention, and long-term business growth?

Based on your analysis, recommend whether the company should implement the smart factory automation system and justify your decision.

Google Collab step – by – step work

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

NumPy (np) Used for creating random numbers and numerical data.

Pandas (pd) Used for working with tables (datasets).

Matplotlib (plt) Used to create graphs and charts.

Step 2 – Set Random Seed

```
np.random.seed(42)

n = 1000

data = {

    "machine_id": np.random.randint(1,50,n),

    "machine_uptime_hours": np.random.normal(20,3,n),

    "downtime_minutes": np.random.randint(5,120,n),

    "production_output_units": np.random.randint(200,500,n),

    "defect_rate_percent": np.random.uniform(0.5,5,n),

    "temperature_celsius": np.random.normal(70,10,n),

    "vibration_level": np.random.uniform(0.1,1.5,n)
}

df = pd.DataFrame(data)

df.head()
```

	machine_id	machine_uptime_hours	downtime_minutes	production_output_units	defect_rate_percent	temperature_celsius	vibration_level
0	39	20.521899	49	486	1.960303	68.884912	0.865067
1	29	22.723616	104	407	1.712803	61.950335	0.227895
2	15	19.973628	98	313	1.359170	66.123001	1.159305
3	43	21.354234	59	221	3.625884	65.863843	0.915409
4	8	22.447555	117	413	1.484220	65.223544	0.361226

Next steps: Generate code with df | New interactive sheet

```
np.random.seed(42)
```

Why do we use this code? This ensures **the random numbers remain the same every time we run the code**.

Without seed: Data changes every run.

With seed: Data stays **consistent and reproducible**.

Example: The teacher runs your notebook → gets the **same results as you**.

Step 3 – Define Dataset Size

```
n = 1000
```

We will generate **1000 machine records**.
Each record represents **machine performance data**.

Step 4 – Create Synthetic Smart Factory Data

```
data = {
```

Here we create **different machine variables**.

machine_id

```
"machine_id": np.random.randint(1,50,n)
```

Generate machine numbers between **1 and 50**.

machine_uptime_hours

```
"machine_uptime_hours": np.random.normal(20,3,n)
```

Normal distribution means:

- Average = **20 hours**
- Variation = **±3 hours**

downtime_minutes

```
"downtime_minutes": np.random.randint(5,120,n)
```

Machine stops for **5 to 120 minutes**.

Downtime occurs due to:

- maintenance
- breakdown
- overheating

production_output_units

```
 "production_output_units": np.random.randint(200,500,n)
```

Each machine produces **200–500 units per shift**. This shows **factory productivity**.

defect_rate_percent

```
"defect_rate_percent": np.random.uniform(0.5,5,n)
```

Defective products between **0.5% – 5%**.

This shows quality control

temperature_celsius

```
"temperature_celsius": np.random.normal(70,10,n)
```

Machine temperature around **70°C**.

If the temperature becomes **too high, there is a risk of machine faults**.

vibration_level

```
"vibration_level": np.random.uniform(0.1,1.5,n)
```

Measures machine vibration.

High vibration means: ⚠️ The machine may fail soon.

Step 5 – Create DataFrame

```
df = pd.DataFrame(data)
```

Why? This converts the data into a table format.

DataFrame = **structured dataset**.

Step 6 – Display First 5 Rows

```
df.head()
```

Shows **first 5 rows of dataset**.

This helps check: data generated correctly, Columns are correct

Step – 7 : Data Summary

```
df.describe()
```

	machine_id	machine_uptime_hours	downtime_minutes	production_output_units	defect_rate_percent	temperature_celsius	vibration_level
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	25.379000	20.256979	61.357000	344.278000	2.724050	69.304827	0.794330
std	14.122138	3.139907	32.510098	85.250668	1.292480	10.085866	0.407683
min	1.000000	9.945168	5.000000	200.000000	0.500138	39.923677	0.105486
25%	13.000000	18.188241	34.000000	269.000000	1.606294	62.629081	0.434205
50%	26.000000	20.254910	60.000000	340.000000	2.716485	69.398851	0.799711
75%	37.000000	22.371012	90.000000	415.000000	3.821809	75.826129	1.155339
max	49.000000	29.502115	119.000000	499.000000	4.998010	102.430930	1.499245

Why we use this? Shows **statistical summary**.

Example:

Statistic	Meaning
mean	average value
min	smallest value
max	highest value
std	variation

This helps understand **machine performance patterns**.

Step 8 – Fault Risk Prediction

```
df["fault_risk"] = np.where(

    (df["temperature_celsius"] > 85) |
    (df["vibration_level"] > 1.2) |
    (df["downtime_minutes"] > 90),

    "High Risk",
    "Normal"
)

df.head()
```

	machine_id	machine_uptime_hours	downtime_minutes	production_output_units	defect_rate_percent	temperature_celsius	vibration_level	fault_risk
0	39	20.521899	49	486	1.960303	68.884912	0.865067	Norma
1	29	22.723616	104	407	1.712803	61.950335	0.227895	High Risk
2	15	19.973628	98	313	1.359170	66.123001	1.159305	High Risk
3	43	21.354234	59	221	3.625884	65.863843	0.915409	Norma
4	8	22.447555	117	413	1.484220	65.223544	0.361226	High Risk

```
df["fault_risk"] = np.where(
```

We create a **new column called fault_risk**.

```
 (df["temperature_celsius"] > 85)
```

If machine temperature > **85°C**

Machine may **overheat**.

```
(df["vibration_level"] > 1.2)
```

High vibration indicates:

⚠️ Mechanical problem.

```
(df["downtime_minutes"] > 90)
```

Long downtime indicates:

⚠️ Possible breakdown.

```
"High Risk"
```

If any condition is true → machine is **High Risk**.

```
 "Normal"
```

Otherwise machine is **Normal**.

Step 9 – View Updated Data

```
df.head()
```

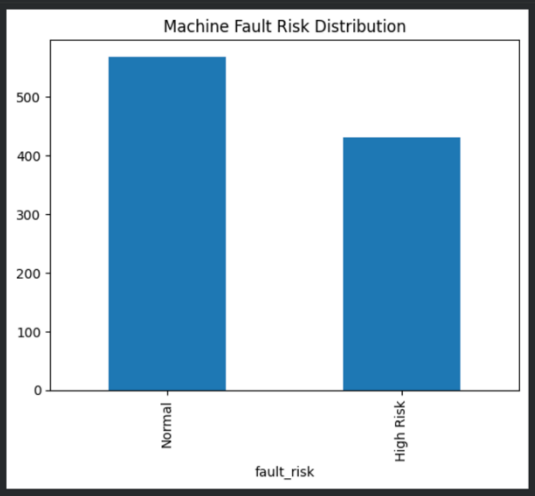
Now dataset includes **fault risk column**.

Step 10 – Count Risk Catgories

```
df["fault_risk"].value_counts().plot(kind="bar")

plt.title("Machine Fault Risk Distribution")

plt.show()
```




```
plt.scatter(df["downtime_minutes"], df["production_output_units"])

plt.xlabel("Downtime Minutes")
plt.ylabel("Production Output")

plt.title("Downtime vs Production")

plt.show()
```

```
df["fault_risk"].value_counts()
```

Counts machines in each category.

Step 11 – Bar Chart

```
df["fault_risk"].value_counts().plot(kind="bar")
```

Creates **bar graph** showing:

- Normal machines
- High risk machines

This helps **visualize machine health**.

Step 12 – Chart Title

```
plt.title("Machine Fault Risk Distribution")
```

Adds graph title.

Step 13 – Show Graph

```
plt.show()
```

Displays the chart.

Step 14 – Scatter Plot

```
plt.scatter(df["downtime_minutes"], df["production_output_units"])
```

Shows relationship between:

- downtime
- production output

This helps analyze:

More downtime → **less production**.

Step 15 – Axis Labels

```
plt.xlabel("Downtime Minutes")
```

X-axis = downtime.

```
plt.ylabel("Production Output")
```

Y-axis = production.

Step 16 – Plot Title

```
plt.title("Downtime vs Production")
```

Displays the graph.

Step 17 – Show Scatter Plot

```
plt.show()
```

Displays the graph.

Step 18 – Save Dataset

```
df.to_csv("smart_factory_machine_data.csv", index=False)
```

Why we use this?

Exports the dataset to **CSV file**.

CSV can be opened in:

- Excel
- Power BI
- Tableau

`index=False`
Removes unnecessary row numbers.